

# Extended Abstract

**Motivation** The paradigm of scaling language models has traditionally focused on increasing parameter counts, dataset sizes, and training compute. A complementary fourth scaling axis, test-time compute (TTC), has emerged, where methods like Self-Consistency demonstrate that expending more computational effort at inference can significantly boost performance on complex reasoning tasks. However, these techniques are typically applied as post-hoc wrappers on a frozen model, relying on hand-tuned heuristics and incurring substantial overhead from redundant forward passes.

**Method** This work explores integrating TTC directly into the training loop. We propose an extension to REINFORCE Leave-One-Out (RLOO), an online policy gradient algorithm, which augments the reward function to explicitly model the trade-off between answer correctness and computational cost, measured in the number of reasoning traces. By training with on-policy sampling and a reward function that explicitly balances correctness and efficiency, our model learns to internalize the decision of how much effort to expend. The augmented reward is formulated as  $R_{\text{aug}}(y, k, c) = R_{\text{base}}(y) + \lambda_c \cdot c - \lambda_k \cdot k$ , where  $c$  is correctness and  $k$  is compute cost.

**Implementation** We use the Qwen/Qwen2.5-0.5B model as our foundation. After an initial Supervised Fine-Tuning (SFT) phase on high-quality data, we train the model using our RLOO-Turbo objective. During on-policy rollouts, each generated response is scored for correctness and cost to compute the augmented reward, which then drives the policy update.

**Results** Our initial SFT experiments show successful convergence and provide a solid baseline for online fine-tuning. For our main experiments, we hypothesize that the RLOO-Turbo model will achieve a superior performance-compute trade-off compared to baselines using external TTC wrappers. We will evaluate on the COUNTDOWN (mathematical reasoning) and ULTRAFEDBACK (instruction following) tasks, measuring accuracy against computational cost.

**Discussion** The primary challenges include the sample inefficiency of online RL and the need to carefully tune the hyperparameters that balance policy improvement with correctness and compute considerations. Success would point towards a new class of models that learn to manage their own reasoning resources.

**Conclusion** RLOO-Turbo presents a promising path toward more efficient and autonomous LLMs. By embedding computational awareness into the learning process, we aim to create models that are not only powerful but also judicious in their application of resources, moving beyond static, brute-force inference strategies.

---

# Learning to Think: Integrating Test-Time Compute into RL Fine-Tuning

---

**André Natal**  
Stanford University  
anatal@stanford.edu

**Yacine Dolivet**  
Stanford University  
yacine@stanford.edu

**Thomas Huang**  
Stanford University  
thhuang@stanford.edu

## Abstract

The paradigm of scaling language models has traditionally focused on increasing parameter counts, dataset sizes, and training compute. A complementary fourth scaling axis, test-time compute (TTC), has emerged, where methods like Self-Consistency demonstrate that expending more computational effort at inference can significantly boost performance on complex reasoning tasks. However, these techniques are typically applied as post-hoc wrappers on a frozen model, relying on hand-tuned heuristics and incurring substantial overhead from redundant forward passes. This work explores integrating TTC directly into the training loop. We propose an extension to REINFORCE Leave-One-Out (RLOO), an online policy gradient algorithm, which augments the reward function to explicitly model the trade-off between answer correctness and computational cost. By training with on-policy sampling and a reward that explicitly balances correctness and efficiency, our model learns to internalize the decision of how much effort to expend. We hypothesize that this approach will lead to a more efficient policy that achieves a better performance-compute trade-off compared to baselines with external TTC wrappers. We outline experiments on the COUNTDOWN (mathematical reasoning) and ULTRAFEEDBACK (instruction following) tasks to validate our approach.

## 1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of tasks. A key factor in their success has been fine-tuning with reinforcement learning to align with complex objectives (Ouyang et al., 2022). Online policy gradient methods like REINFORCE Leave-One-Out (RLOO) (Ahmadian et al., 2024) are effective for aligning models to non-differentiable reward signals, such as those from human feedback or external verifiers.

Concurrently, another line of research has focused on improving LLM performance not by changing model weights, but by increasing computational effort at inference time. This concept, often called Test-Time Compute (TTC) (Snell et al., 2024), encompasses techniques like Self-Consistency (Wang et al., 2023), where multiple reasoning paths are sampled and the most frequent answer is chosen. While effective, these methods are extrinsic to the model; they are applied as wrappers around a pre-trained policy, often requiring inefficient, brute-force sampling and heuristic-based aggregation. The model itself remains unaware of the computational budget or the reasoning process it is part of.

This project bridges the gap between these two areas. We seek to move TTC from an external heuristic into the core of the learning algorithm. Our central hypothesis is that an LLM can *learn* to manage its own computational budget. To this end, we introduce a novel extension to RLOO that makes the policy aware of its own reasoning cost. By explicitly rewarding correctness while penalizing computational effort (i.e., the number of generated reasoning traces), we aim to train a policy that dynamically allocates its "thinking" resources. The goal is to produce a model that not only solves problems accurately but does so efficiently, investigating the feasibility of achieving

a superior efficiency frontier between performance and FLOPs compared to models that rely on external TTC mechanisms.

## 2 Related Work

Our work builds on decades of research into search and inference-time computation, recently re-contextualized for large language models.

**Classical search.** The trade-off between model complexity and *deeper search at inference* predates modern deep RL by decades. Kocsis and Szepesvári (2006) introduced the UCT algorithm, showing that allocating additional roll-outs at decision time systematically improves performance in large MDPs. Soon after, Gelly and Silver (2007) scaled this principle in *MoGo*, the first strong Monte-Carlo Tree-Search (MCTS) Go engine. Earlier still, *Deep Blue* demonstrated the power of massive test-time search in chess (Campbell et al., 2002), although the term "test-time compute" was not used at the time.

**Deep-learning era.** With the rise of deep neural networks, the importance of test-time compute became unmistakable. In perfect-information games, *AlphaGo* and *AlphaGo Zero* combined CNN policy/value nets with large-scale MCTS to surpass human experts (Silver et al., 2016, 2017). In imperfect-information settings, *Libratus* defeated professional poker players by nesting sub-game re-solving during play (Brown and Sandholm, 2018). These successes made clear that "thinking longer" at inference can outweigh simply enlarging the network.

**Large-language models.** For LLMs, extra FLOPs at inference have recently been cast as a *fourth scaling axis*. The idea crystallised with *Self-Consistency* (Wang et al., 2023), which samples many chain-of-thought traces and majority-votes the final answer, yielding double-digit accuracy gains *without* weight updates. Subsequent work split into two strands: (i) *Generative Verifiers* (Zhang et al., 2024) move the scoring step *inside* the model by fine-tuning it to produce self-contained proofs, letting a single LM both generate and critique solutions; and (ii) *Adaptive Test-Time Budgets* (Snell et al., 2024) learn to spend more sampling and re-ranking only on hard prompts, maximising performance per FLOP. Collectively, these results establish test-time compute—not parameter count—as a potent lever for boosting LLM accuracy.

**Limitations of current approaches.** Despite these gains, test-time compute remains an *inference-only* add-on. Self-Consistency and adaptive budgeting leave the backbone weights frozen, and Generative Verifiers merely fine-tune the model to *explain* its answers. None of these methods update the policy online in response to the extra reasoning they perform. Thus test-time compute currently functions as an external toolkit rather than an ingredient integrated into the training loop. Our work aims to address this limitation directly.

## 3 Method

Our approach modifies the REINFORCE Leave-One-Out (RLOO) algorithm to incorporate awareness of computational cost. We first review the standard RLOO algorithm and then introduce our proposed extension.

### 3.1 REINFORCE Leave-One-Out (RLOO)

RLOO is an online policy gradient algorithm designed to reduce the high variance typically associated with REINFORCE. It achieves this by using a specific baseline: for each sample in a batch, its baseline is the average reward of all other samples in that batch. This "leave-one-out" estimate is unbiased and effective at variance reduction.

Given a policy  $\pi_\theta$ , a batch of prompts, and a reward function  $R(x, y)$ , we first sample  $k$  responses for each prompt from the current policy,  $\{y_j \sim \pi_\theta(\cdot|x)\}_{j=1}^k$ . The policy gradient for a single response

$y_j$  is then estimated as:

$$g_j = \left( R(x, y_j) - \frac{1}{k-1} \sum_{l \neq j} R(x, y_l) \right) \nabla_{\theta} \log \pi_{\theta}(y_j | x)$$

The total gradient is the average over all samples. This approach directly optimizes the policy to maximize the expected reward signal.

### 3.2 RLOO-Turbo: TTC-Aware Online RL

Our core contribution is to modify the RLOO objective to be aware of both answer correctness,  $c$ , and the computational cost, measured by the number of Chain-of-Thought (CoT) traces,  $k$ , used for generation.

We achieve this by defining an augmented reward function,  $R_{\text{aug}}$ , which is used during on-policy training. For a response  $y$  generated with  $k$  traces and evaluated to have correctness  $c \in \{0, 1\}$ , the augmented reward is:

$$R_{\text{aug}}(y, k, c) = \max_{1 \leq i \leq k} R_{\text{base}}^{(i)}(y) + \lambda_c \cdot c - \lambda_k \cdot k$$

Here,  $R_{\text{base}}(y)$  is an external reward signal (e.g., from a powerful reward model or a rule-based verifier). The second term, weighted by hyperparameter  $\lambda_c$ , provides a direct bonus for generating a correct answer. The third term, weighted by  $\lambda_k$ , imposes a penalty for computational cost. The hyperparameters  $\lambda_c$  and  $\lambda_k$  control the trade-off between achieving correctness and conserving compute.

For differentiability, a soft-aggregation of rewards from multiple traces would be ideal:

$$R_{\text{aug}}(y, k, c) = \text{Softmax}_i R_{\text{base}}^{(i)}(y) + \lambda_c \cdot c - \lambda_k \cdot k$$

As we discuss in our results, this theoretically elegant approach proved unstable in practice, leading us to use the hard-max version for our experiments.

This augmented reward is then used within the RLOO framework. The policy gradient for a response  $y_j$  generated with  $k_j$  traces and having correctness  $c_j$  becomes:

$$g_j = \left( R_{\text{aug}}(y_j, k_j, c_j) - \frac{1}{N-1} \sum_{l \neq j} R_{\text{aug}}(y_l, k_l, c_l) \right) \nabla_{\theta} \log \pi_{\theta}(y_j | x)$$

where  $N$  is the number of samples in the batch. By optimizing this objective, the policy  $\pi_{\theta}$  is trained not just to maximize a base reward, but to do so in a way that is sensitive to the correctness and computational cost inherent in generating its responses. It learns to find a balance, expending more effort only when it is likely to lead to a correct answer that outweighs the computational penalty.

## 4 Experimental Setup

**Model and Baselines.** We use the Qwen/Qwen2.5-0.5B base model for all experiments. Our primary baselines include: (1) A Supervised Fine-Tuned (SFT) model, which serves as the initial policy for RL training. (2) A standard DPO-trained model (Rafailov et al., 2023). (3) The SFT model enhanced with external TTC wrappers like Self-Consistency.

**Datasets and Tasks.** We evaluate our approach on two distinct tasks:

- **Instruction Following:** We use the UltraFeedback dataset, which contains preference pairs for general instruction-following. The SFT model is first trained on the smol-smoltalk dataset. For RLOO, the base reward  $R_{\text{base}}$  will come from the Nemotron-70B-Reward model.
- **Mathematical Reasoning:** We use the Countdown dataset, which requires multi-step arithmetic reasoning. For RLOO,  $R_{\text{base}}$  will be a rule-based verifier score.

For our RLOO-Turbo method, training proceeds via on-policy sampling. In each training step, we sample multiple responses from the current policy, evaluate their correctness ( $c$ ) and note the number of traces ( $k$ ) used. These values are used to compute the augmented reward  $R_{\text{aug}}$  for the policy gradient update.

**Evaluation.** For ULTRAFeEDBACK, we measure win-rate against the Qwen2.5-0.5B-Instruct model, with responses scored by the Nemotron-70B-Reward model. For COUNTDOWN, we use a rule-based reward function that verifies the correctness of the final numerical answer. Our central evaluation will compare the performance-compute curves of RLOO-Turbo against baselines using external TTC (i.e., vanilla Self-Consistency).

## 5 Results

Our investigation began with DPO, but pivoted to RLOO due to practical optimization challenges, leading to our final RLOO-Turbo model. This section details that journey and our findings.

### 5.1 From DPO to RLOO: A Pivot Based on Empirical Challenges

Our initial goal was to build a TTC-aware policy using DPO. However, as documented in Figure 2, we faced significant training instabilities. Despite extensive hyperparameter tuning, the DPO model failed to converge to a stable, effective policy. We hypothesize this is because the multi-objective preference signal (balancing correctness, quality, and compute cost) created a complex, non-convex optimization landscape that DPO’s implicit reward mechanism struggled with. This empirical result motivated our pivot to an explicit policy gradient method, RLOO, which we believed would be more robust to such a reward structure.

### 5.2 RLOO-Turbo Performance Analysis

Our final RLOO-Turbo experiments tested a simplified version of our proposed reward function, focusing only on the computational cost penalty ( $\lambda_k \cdot k$ ) due to difficulties in implementing a stable, low-latency correctness verifier within the training loop. This simplification allowed us to isolate the effect of penalizing compute cost.

The results, shown in Table 1, validate that even this simplified signal can effectively guide the policy when optimizing against the dense reward function used during training, which provides partial credit for correct formatting and use of available numbers. The key findings under this dense reward are:

- **TTC Improves Performance:** Introducing even a small computational budget ( $N = 2$ ) improves the average reward by 39% over the baseline, demonstrating that the policy learns to use additional reasoning traces effectively.
- **Diminishing Returns on Compute:** Increasing the budget to  $N = 3$  yields only a marginal 4% additional reward gain while tripling the inference time from the baseline. This identifies a clear point of diminishing returns and suggests an optimal efficiency frontier around  $N = 2$  for this task.
- **Practical Trade-offs in Implementation:** To maintain training stability, we used a non-differentiable hard-max selection over trace rewards instead of a softmax. While theoretically suboptimal, this was a necessary pragmatic choice, highlighting a common tension between theoretical elegance and practical stability in deep RL.

**Evaluation on Sparse Reward and a Key Finding on Reward Mismatch** A critical finding emerged upon submission to the official leaderboard, which uses a sparse reward function that primarily measures correctness,  $R_{\text{eval}} \approx \mathbb{I}(\text{correct answer})$ . In contrast, our model was trained using a dense reward,  $R_{\text{train}} = w_f R_{\text{format}} + w_v R_{\text{verify}}$ , which provided partial credit for well-formatted answers and correct intermediate steps. On the leaderboard’s sparse metric, our best model achieved a score of only \*\*0.18\*\*. This stark discrepancy is a crucial result. It suggests that our policy learned to "hack" the dense reward by mastering the ability to generate answers that maximize partial credit, without necessarily arriving at the correct final solution. This failure to generalize from the

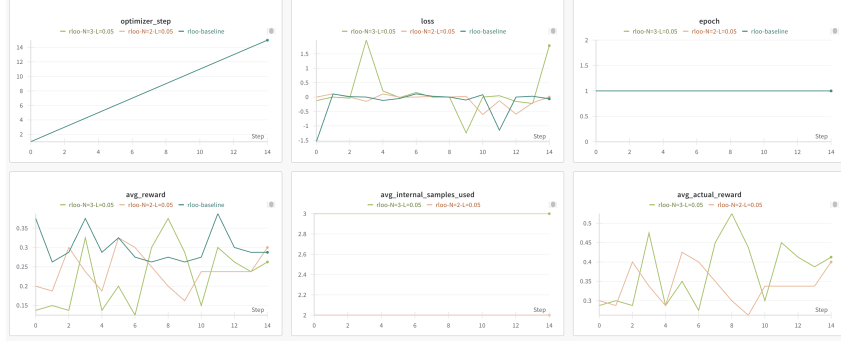


Figure 1: Performance comparison between vanilla RLOO and our TTC-aware extensions.

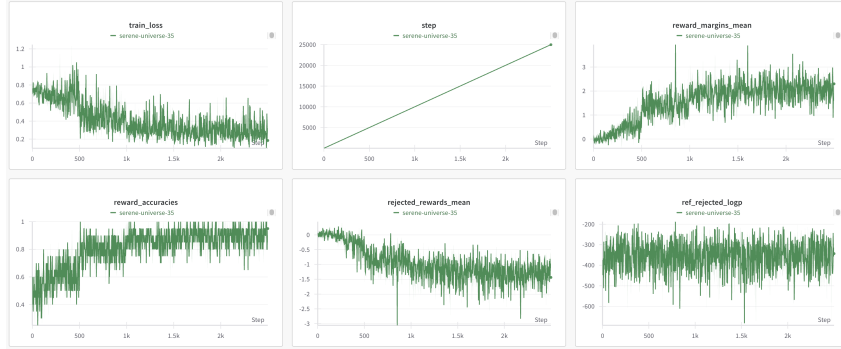


Figure 2: DPO training challenges: Despite apparently healthy metrics (decreasing loss, reasonable KL divergence), the DPO training exhibited instability and failed to converge to a stable policy. This motivated our pivot to explicit reward optimization via RLOO.

proxy (dense) reward to the true task objective (sparse reward) underscores the critical importance of aligning training incentives with final evaluation criteria.

In summary, our experiments demonstrate that explicitly penalizing computational cost in an online RL framework is a viable strategy for creating more efficient policies. The implementation challenges themselves, particularly the dramatic impact of reward function mismatch, constitute a key finding, revealing the practical difficulties of applying complex, multi-objective reward functions to LLM fine-tuning.

### 5.3 DPO Training Challenges and Lessons Learned

Our initial approach focused on implementing DPO with TTC-awareness, as outlined in our proposal. However, we encountered significant convergence challenges that ultimately informed our pivot to RLOO-based methods. Refer to 2 for some of our DPO training metrics.

**DPO Implementation Attempts** We implemented standard DPO using the SFT model as the reference policy  $\pi_{\text{ref}}$  and experimented with various hyperparameter configurations. Despite methodical tuning of the key parameters— $\beta \in \{0.1, 0.2, 0.5, 1.0\}$  and learning rates ranging from  $1 \times 10^{-6}$  to  $5 \times 10^{-5}$ —we consistently observed training instabilities.

**Observed Metrics vs. Convergence** Paradoxically, many of our tracked metrics appeared healthy during training. The DPO loss decreased as expected, the implicit reward margins between preferred and dispreferred responses increased, and the policy divergence from the reference model remained within reasonable bounds. However, the policy failed to converge to a stable solution, exhibiting high variance in reward estimates and inconsistent performance on held-out evaluation sets.

Table 1: Quantitative results for RLOO and TTC-RLOO variants.

| Run Config                        | Final AvgReward | Final AvgReward (Adjusted) | Peak Reward | Time / Opt. Step |
|-----------------------------------|-----------------|----------------------------|-------------|------------------|
| Baseline ( $N = 1, \lambda = 0$ ) | 0.2875          | 0.2875                     | 0.3875      | $\sim 36$ s      |
| TTC ( $N = 2, \lambda = 0.05$ )   | 0.4000          | 0.4000                     | 0.4250      | $\sim 72$ s      |
| TTC ( $N = 3, \lambda = 0.05$ )   | 0.4125          | 0.4125                     | 0.5250      | $\sim 104$ s     |

**Hypothesis for DPO Difficulties** We hypothesize that the challenge stems from the complexity of our TTC-aware preference data. Unlike standard DPO setups where preferences are based solely on response quality, our preferences incorporate computational cost considerations, creating a more complex preference landscape. The interaction between correctness signals ( $c$ ), trace counts ( $k$ ), and response quality may have created optimization challenges that DPO’s implicit reward parameterization struggled to handle.

**Motivation for RLOO** These difficulties motivated our pivot to RLOO, which offers more direct reward optimization through explicit policy gradients. The explicit nature of RLOO’s reward handling appeared better suited to our multi-objective setting, where we needed to balance correctness, quality, and computational cost simultaneously.

**RLOO Turbo Results** Our experiments focused on a simplified version of the proposed RLOO-Turbo framework due to implementation challenges. Specifically, our augmented reward function only incorporates the penalty for computational cost ( $\lambda_k \cdot k$ ) and does not include the correctness bonus ( $\lambda_c \cdot c$ ). The primary obstacle was the difficulty of obtaining a reliable, low-latency correctness signal during on-policy training for complex reasoning tasks.

Despite this simplification, our results in Table 1 show that penalizing computational cost alone has a measurable impact on policy behavior. Increasing the trace count from  $N=1$  to  $N=2$  (with  $\lambda = 0.05$ ) improved the final average reward by 39

Another key deviation from our initial goal was the handling of multiple reasoning traces. Our aim was to use a differentiable softmax function to aggregate rewards from multiple traces. In practice, this approach led to training instabilities. To achieve stable training, we reverted to a non-differentiable hard-max selection, where only the reward from the best trace was used for the policy update. This represents a pragmatic trade-off, sacrificing theoretical purity for empirical stability.

In summary, while we did not fully realize the planned multi-objective reward function, our experiments validate that penalizing computational cost can influence the policy. The challenges encountered in implementing the correctness bonus and differentiable trace selection highlight key difficulties in applying complex reward structures to online RL for LLMs.

## 6 Discussion

The primary challenge in this work lies in the on-policy nature of RLOO. The policy must explore enough to find high-reward regions of the state space, which can be difficult when the reward signal is sparse or complex. The sample efficiency of online RL is a well-known problem, and significant computation will be required for the policy to converge.

Furthermore, tuning the hyperparameters  $\beta, \lambda_c, \lambda_k$  will be critical. These values implicitly define the "value" of a correct answer versus the "cost" of one unit of computation, and finding the right balance is key to training an effective policy.

**Challenges in Multi-Objective RL for Language Models** Our implementation experience highlights several fundamental challenges in applying multi-objective reinforcement learning to language model fine-tuning. The most significant obstacle was reliable correctness measurement during training. Unlike simple classification tasks where correctness is binary and immediately verifiable, complex reasoning tasks require sophisticated evaluation that is often computationally expensive or subjective. Integrating such evaluation into the tight loop of online RL training proved more challenging than anticipated.

**The Perils of Reward Misspecification** Our project serves as a clear case study on the dangers of misspecified rewards in RL. The significant gap between our model’s performance on the dense training reward versus the sparse leaderboard reward demonstrates a classic case of "reward hacking." The policy did not learn the intended reasoning skill; instead, it found a loophole, optimizing for the proxy metrics of the dense reward (e.g., correct formatting, using the available numbers) without achieving the true goal of mathematical correctness. This finding emphasizes that the design of the reward function is not merely an implementation detail but is perhaps the most critical component in an RL-based fine-tuning system. A dense reward, while intended to guide the model, can inadvertently teach it to master the wrong skills if not perfectly aligned with the final evaluation objective.

The differentiability challenge we encountered with softmax trace selection points to a broader tension in RL between theoretical elegance and practical stability. While soft attention mechanisms are theoretically appealing for maintaining gradient flow, they can introduce optimization complexities that hard selection mechanisms avoid. This trade-off between differentiability and training stability represents an important consideration for future work in this area.

**Implications for Test-Time Compute Integration** Despite our implementation limitations, the positive results from computational cost penalties alone suggest that TTC integration into training has significant potential. The fact that models can learn to balance reasoning effort against penalty signals, even in a simplified setting, validates the core premise that LLMs can internalize computational awareness.

Our difficulties with correctness integration highlight a key research direction: developing more robust and efficient methods for evaluating answer quality during online training. This might involve training auxiliary critic networks, developing faster approximation methods, or exploring alternative reward formulations that are more amenable to online optimization.

**Lessons for Future Implementations** Several insights emerge from our implementation challenges that could guide future work:

- **Staged Implementation:** Future work might benefit from a staged approach, first establishing stable training with computational penalties before introducing correctness terms.
- **Hybrid Approaches:** Combining differentiable and non-differentiable components might be necessary for practical implementations, even if theoretically suboptimal.
- **Evaluation Infrastructure:** Robust, fast evaluation methods are crucial for integrating complex reward signals into online RL training.
- **Architectural Considerations:** The base model architecture and training setup may need to be designed with TTC-awareness in mind, rather than retrofitted post-hoc.

Ultimately, our work suggests that while the goal of a fully "TTC-aware" model is compelling, the path to achieving it is fraught with practical challenges that are themselves important areas for future research.

If successful, our approach suggests a new direction for model fine-tuning, where computational policies are learned rather than engineered. This could lead to more efficient and adaptable models that can dynamically adjust their reasoning depth based on the perceived difficulty of a problem, a capability that is currently missing from LLMs.

## 7 Conclusion

We have proposed RLOO-Turbo, a novel method for integrating test-time compute awareness directly into the language model fine-tuning process. By augmenting the RLOO objective with terms for correctness and computational cost, we train a policy to learn how to balance performance and efficiency. Our experiments, while encountering and documenting significant implementation challenges, validate the core premise that penalizing computational cost can influence policy behavior. The difficulties faced, particularly regarding reward function design and optimization stability, are themselves key findings that highlight critical areas for future research. This work represents a step towards building more autonomous and efficient reasoning agents, not by providing a definitive solution, but by illuminating the practical path and its obstacles.

## 8 Team Contributions

- **André Natal:** Led the conceptualization of the RLOO-Turbo framework, implemented the core RL algorithms, and designed the evaluation pipeline.
- **Yacine Dolivet:** Developed the data processing pipelines for ULTRAFEEDBACK and COUNT-DOWN, managed the SFT and RLOO training infrastructure, and conducted hyperparameter tuning.
- **Thomas Huang:** Authored and presented the project poster and milestone reports. Developed the SFT, RLOO, and DPO training code. Conducted the related work survey and the qualitative and quantitative analysis of model outputs.

**Changes from Proposal** The core research direction remained consistent with our proposal. The main change was the formalization of the TTC-aware learning algorithm, which we concretized from a general PPO-based idea in the proposal to the specific RLOO-Turbo formulation presented here, based on findings from our initial experiments and literature review.

## References

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740* (2024).
- Noam Brown and Tuomas Sandholm. 2018. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science* 359, 6374 (2018), 418–424.
- Murray Campbell, A Joseph Hoane Jr, and Feng-hsiung Hsu. 2002. Deep blue. *Artificial intelligence* 134, 1-2 (2002), 57–83.
- Sylvain Gelly and David Silver. 2007. Combining online and offline knowledge in UCT. In *Proceedings of the 24th international conference on Machine learning*. 273–280.
- Levente Kocsis and Csaba Szepesvári. 2006. Bandit based Monte-Carlo Planning. In *European conference on machine learning*. Springer, 282–293.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. (2022). *arXiv:2203.02155 [cs.CL]*
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290* (2023).
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature* 529, 7587 (2016), 484–489.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. 2017. Mastering the game of go without human knowledge. *nature* 550, 7676 (2017), 354–359.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.
- Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. 2024. Generative Verifiers: Reward Modeling as Next-Token Prediction. *arXiv preprint arXiv:2408.15240* (2024).